

# Postgres-Backed Async Job Queue

Design Spec + Production Implementation Guide — v1

Core Use Case: Scheduled report generation (weekly complaint report -> hotel manager email) | Stack: Python + PostgreSQL (no Redis)

| Property           | Decision                                  |
|--------------------|-------------------------------------------|
| Broker             | PostgreSQL (jobs table)                   |
| Concurrency Safety | SELECT ... FOR UPDATE SKIP LOCKED         |
| Durability         | All state in Postgres, crash-safe         |
| Retries            | attempts < max -> run_at + 1h backoff     |
| Stale Lock Timeout | 15 minutes (configurable)                 |
| Scheduler          | Separate process, only enqueues job_type  |
| Scalability        | Add worker processes -> linear throughput |

## 1. Job Storage — Schema

Single table is source of truth. JSONB payload gives flexibility. Partial indexes keep polling fast.

```
-- migrations/001_create_jobs.sql
CREATE TYPE job_status AS ENUM ('pending', 'processing', 'completed', 'failed');

CREATE TABLE jobs (
  id BIGSERIAL PRIMARY KEY,
  job_type TEXT NOT NULL,
  payload JSONB NOT NULL DEFAULT '{} '::jsonb,
  status job_status NOT NULL DEFAULT 'pending',
  attempts INT NOT NULL DEFAULT 0,
  max_attempts INT NOT NULL DEFAULT 3,
  run_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  locked_at TIMESTAMPTZ,
  locked_by TEXT,
  last_error TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_jobs_poll ON jobs (status, run_at, id)
  WHERE status = 'pending';
CREATE INDEX idx_jobs_stale ON jobs (status, locked_at)
  WHERE status = 'processing';
CREATE INDEX idx_jobs_type ON jobs (job_type);

-- updated_at trigger
CREATE OR REPLACE FUNCTION touch_updated_at() RETURNS TRIGGER AS
$fn$ BEGIN NEW.updated_at = NOW(); RETURN NEW; END; $fn$ LANGUAGE plpgsql;

CREATE TRIGGER trg_jobs_updated_at
BEFORE UPDATE ON jobs FOR EACH ROW EXECUTE FUNCTION touch_updated_at();
```

Lifecycle: pending -> processing -> completed | failed. Retry path: failed attempt -> pending with future run\_at.

## 2. Job Submission

```
# app/queue.py
import psycopg2
from psycopg2.extras import Json

def enqueue(conn, job_type: str, payload: dict, run_at=None, max_attempts=3):
    with conn.cursor() as cur:
        cur.execute(
            "INSERT INTO jobs (job_type, payload, run_at, max_attempts) "
            "VALUES (%s, %s, COALESCE(%s, NOW()), %s) RETURNING id, run_at",
            (job_type, Json(payload), run_at, max_attempts)
        )
        job_id, scheduled_at = cur.fetchone()
    conn.commit()
    return job_id

# Example
# enqueue(conn, 'generate_weekly_report',
#         {'hotel_id': 42, 'email': 'mgr@hotel.com', 'week': '2026-W36'})
```

## 3. Worker — Atomic Claim with SKIP LOCKED

Core invariant: no double-processing. Claim and status change in single transaction. SKIP LOCKED lets concurrent workers skip locked rows.

```
# app/worker.py
import os, time, traceback, psycopg2
WORKER_ID = f"{os.uname().nodename}-{os.getpid()}"
POLL_INTERVAL = 2

CLAIM_SQL = '''
WITH claimed AS (
    SELECT id FROM jobs
    WHERE status='pending' AND run_at <= NOW()
    ORDER BY run_at ASC, id ASC LIMIT 1
    FOR UPDATE SKIP LOCKED
)
UPDATE jobs SET status='processing',
    locked_at=NOW(), locked_by=%s, attempts=attempts+1
FROM claimed WHERE jobs.id=claimed.id
RETURNING jobs.id, jobs.job_type, jobs.payload, jobs.attempts, jobs.max_attempts;
'''

def claim_job(conn):
    with conn.cursor() as cur:
        cur.execute(CLAIM_SQL, (WORKER_ID,))
        row = cur.fetchone()
    conn.commit()
    return row

def complete_job(conn, job_id):
    with conn.cursor() as cur:
        cur.execute("UPDATE jobs SET status='completed', locked_at=NULL WHERE id=%s", (
            job_id,))
    conn.commit()

def fail_job(conn, job_id, error, attempts, max_attempts):
    with conn.cursor() as cur:
        if attempts < max_attempts:
            cur.execute(
                "UPDATE jobs SET status='pending', run_at=NOW()+INTERVAL '1 hour', "
                "last_error=%s, locked_at=NULL, locked_by=NULL WHERE id=%s",
                (error[:4000], job_id))
```

```

    )
    else:
        cur.execute(
            "UPDATE jobs SET status='failed', last_error=%s, locked_at=NULL,
locked_by=NULL WHERE id=%s",
            (error[:4000], job_id)
        )
        trigger_alert(job_id, error)
        conn.commit()

def trigger_alert(job_id, error):
    print(f"[ALERT] Job {job_id} permanently failed: {error}")

def run_worker(dsn, registry):
    conn = psycopg2.connect(dsn)
    conn.autocommit = False
    print(f"Worker {WORKER_ID} started")
    while True:
        try:
            job = claim_job(conn)
            if not job:
                time.sleep(POLL_INTERVAL); continue
            job_id, job_type, payload, attempts, max_attempts = job
            handler = registry.get(job_type)
            if not handler:
                fail_job(conn, job_id, f"No handler for {job_type}", attempts,
max_attempts); continue
            try:
                handler(payload)
                complete_job(conn, job_id)
                print(f"Completed {job_id} [{job_type}]")
            except Exception as e:
                err = traceback.format_exc()[-4000:]
                print(f"Job {job_id} failed attempt {attempts}: {e}")
                fail_job(conn, job_id, err, attempts, max_attempts)
        except Exception as e:
            print(f"Loop error: {e}"); traceback.print_exc()
            conn.rollback(); time.sleep(2)
            conn = psycopg2.connect(dsn)

```

Why this satisfies NFRs: No dropped jobs (row stays in processing if crash, recovered by sweeper). No duplicates (row-level lock). Horizontally scalable (run same worker code in N processes).

## 4. Stale Lock Recovery

```

# app/recovery.py
STALE_SQL = '''
UPDATE jobs SET status='pending', locked_at=NULL, locked_by=NULL,
    last_error=COALESCE(last_error,'') || E'\n[recovered stale at ' || NOW() || ']'
WHERE status='processing' AND locked_at < NOW() - INTERVAL '15 minutes'
RETURNING id;
'''

def recover_stale(conn, timeout_min=15):
    # timeout_min is embedded safely as int
    sql = STALE_SQL.replace('15 minutes', f'{int(timeout_min)} minutes')
    with conn.cursor() as cur:
        cur.execute(sql)
        rows = cur.fetchall()
    conn.commit()
    if rows:
        print(f"Recovered {len(rows)} stale jobs: {[r[0] for r in rows]}")
    return rows

```

```
# Run as: cron every minute OR inside worker loop every 30 polls
```

## 5. Scheduler — Separation of Concerns

Scheduler does NOT generate reports. It only knows it's time to enqueue `generate_weekly_report`.

```
# app/scheduler.py
import time, schedule, psycopg2
from app.queue import enqueue
DSN = "dbname=hotel_reports user=postgres"

def enqueue_weekly():
    conn = psycopg2.connect(DSN)
    try:
        enqueue(conn, 'generate_weekly_report', {'week': 'auto', 'trigger':
'cron_mon_6am'})
        print("Enqueued weekly report")
    finally:
        conn.close()

schedule.every().monday.at("06:00").do(enqueue_weekly)

# Alternative production option: pg_cron inside Postgres
# SELECT cron.schedule('weekly-report', '0 6 * * 1',
# $$INSERT INTO jobs(job_type) VALUES ('generate_weekly_report')$$);

if __name__ == '__main__':
    while True:
        schedule.run_pending()
        time.sleep(30)
```

Prod tip: Use Kubernetes CronJob or systemd timer that runs a tiny script calling `enqueue()`. Keeps scheduler stateless and highly available.

## 6. Job Handler Registry

```
# app/registry.py
from app.handlers import reports
REGISTRY = {
    'generate_weekly_report': reports.generate_weekly_report,
    'send_email': reports.send_email,
    'cleanup_old_jobs': reports.cleanup_old_jobs,
}
def get(job_type):
    return REGISTRY.get(job_type)

# app/handlers/reports.py
def generate_weekly_report(payload: dict):
    hotel_id = payload.get('hotel_id')
    print(f"Generating report for hotel {hotel_id}")
    # 1. Query complaints WHERE created_at in week
    # 2. Build PDF
    # 3. Optionally enqueue send_email job
    # from app.queue import enqueue; enqueue(conn, 'send_email', {...})

def send_email(payload: dict):
    # SMTP / SES integration
    print(f"Sending to {payload['email']}")

def cleanup_old_jobs(payload: dict):
    # DELETE FROM jobs WHERE status IN ('completed', 'failed')
    # AND created_at < NOW() - INTERVAL '30 days'
    pass
```

## 7. Non-Functional Requirements Mapping

| NFR                     | How Implemented                                                                                                                                                 |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Durability              | Postgres transaction for every state change. Crash leaves job in processing, recovered by sweeper.                                                              |
| No duplicate processing | FOR UPDATE SKIP LOCKED is row-level lock. Only one worker gets row per transaction.                                                                             |
| Debuggable              | last_error stores traceback (4000 chars), locked_by, attempts. Query: SELECT * FROM jobs WHERE status='processing' AND locked_at < NOW()-INTERVAL '15 minutes'; |
| Horizontally scalable   | Workers stateless. Add processes/containers with same DSN. Throughput ~ N * (1/avg_job_time).                                                                   |

## Operational SQL (no UI needed for v1)

```
-- Queue depth
SELECT job_type, COUNT(*) FROM jobs WHERE status='pending' GROUP BY job_type;

-- Failures last 24h
SELECT id, job_type, attempts, left(last_error,200), created_at
FROM jobs WHERE status='failed' AND created_at > NOW()-INTERVAL '1 day';

-- Retry failed
UPDATE jobs SET status='pending', run_at=NOW(), attempts=0 WHERE id=123;

-- Oldest pending latency (minutes)
SELECT EXTRACT(EPOCH FROM (NOW()-MIN(run_at)))/60 FROM jobs WHERE status='pending';

-- Stuck processing
SELECT * FROM jobs WHERE status='processing' AND locked_at < NOW()-INTERVAL '15
minutes';
```

## 8. Project Structure & Runbook

```
hotel-queue/
  app/
    queue.py      # enqueue()
    worker.py     # claim loop
    recovery.py   # stale sweeper
    scheduler.py  # cron enqueueer
    registry.py   # job_type -> fn
    handlers/reports.py
  migrations/001_create_jobs.sql
  requirements.txt # psycpg2-binary, schedule, python-dotenv

# Setup
createdb hotel_reports
psql hotel_reports < migrations/001_create_jobs.sql

# Run
# Terminal 1: worker (run 3x to test concurrency)
python -m app.worker

# Terminal 2: scheduler
python -m app.scheduler

# Terminal 3: manual enqueue
python -c "import psycpg2; from app.queue import enqueue; c=psycpg2.connect(
'dbname=hotel_reports'); enqueue(c,'generate_weekly_report',{'hotel_id':1})"

# Prod: Dockerfile CMD python -m app.worker, replicas 2-4, 1 scheduler
```

## 9. Out of Scope for v1 (Explicit)

- Redis/other broker — Postgres handles <1k jobs/min fine. Partition if you need >5k/min.
- Distributed coordination beyond Postgres locks — No ZooKeeper/etcd.
- Priority queues / dependencies — All FIFO by run\_at. Job B after A: enqueue B at end of A handler.
- Web dashboard — Use SQL queries above. v2 can add FastAPI view.

## 10. v2 Extensions

- Priority: ADD COLUMN priority INT DEFAULT 0, ORDER BY priority DESC, run\_at ASC
- Exponential backoff: run\_at = NOW() + (2^attempts \* interval '1 min')
- Batch claim LIMIT 10 for throughput
- Metrics: Prometheus counter on completed/failed
- Pydantic validation per job\_type payload

This spec is ready to copy-paste. All SQL is Postgres 14+ compatible, Python 3.10+.